

```
void shift_right(int n)
```

```
{
```

```
    LinkNode* current_r = r;
```

```
    for (int i = 0; i < n; ++i)
```

```
    {
```

```
        if (r->link != nullptr) {
```

```
            current_r = r->link;
```

```
            r->link = l;
```

```
            l = r;
```

```
            r = current_r;
```

```
        }
```

```
    else {
```

```
        l = r;
```

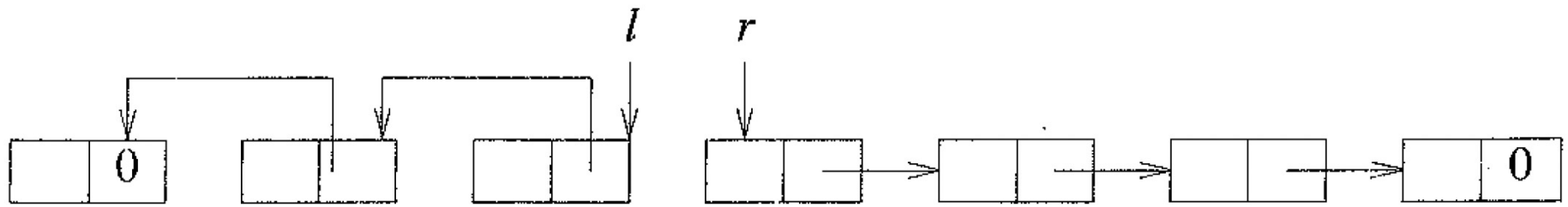
```
        r = nullptr;
```

```
        break;
```

```
    }
```

```
}
```

```
}
```



```
void shift_right(int n)
```

```
{
```

```
    LinkNode* current_r = r;
```

```
    for (int i = 0; i < n; ++i)
```

```
    {
```

```
        if (r->link != nullptr) {
```

```
            current_r = r->link;
```

```
            r->link = l;
```

```
            l = r;
```

```
            r = current_r;
```

```
        }
```

```
    } else {
```

```
        l = r;
```

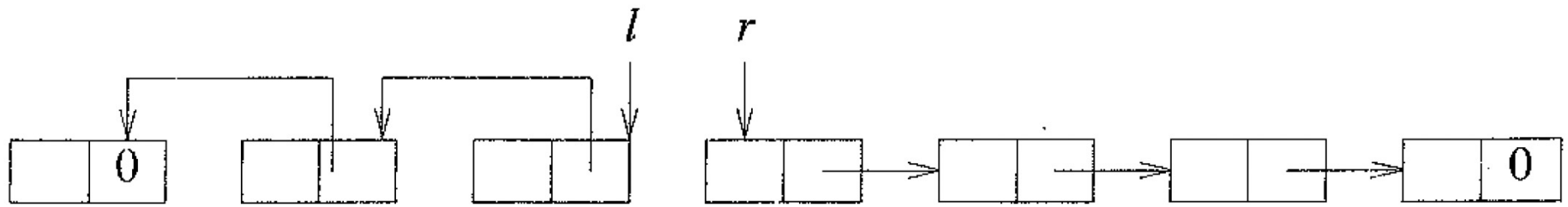
```
        r = nullptr;
```

```
        break;
```

```
    }
```

```
}
```

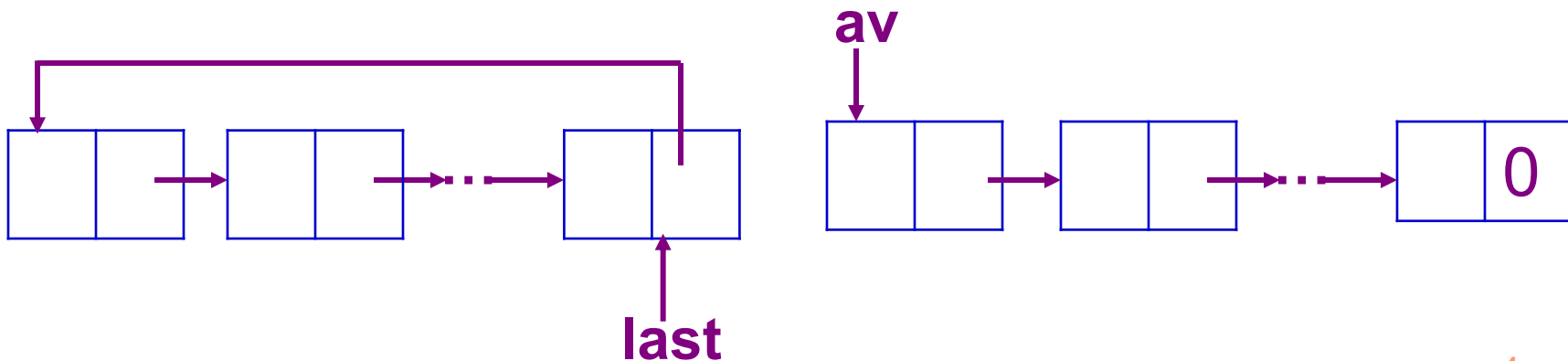
```
}
```



```
void shift_right(int n)
{
    int i = 0;
    while (r & i < n)
    {
        LinkNode* r_next = r->next;
        r->next = l;
        l = r;
        r = r_next; i++;
    }
}
```

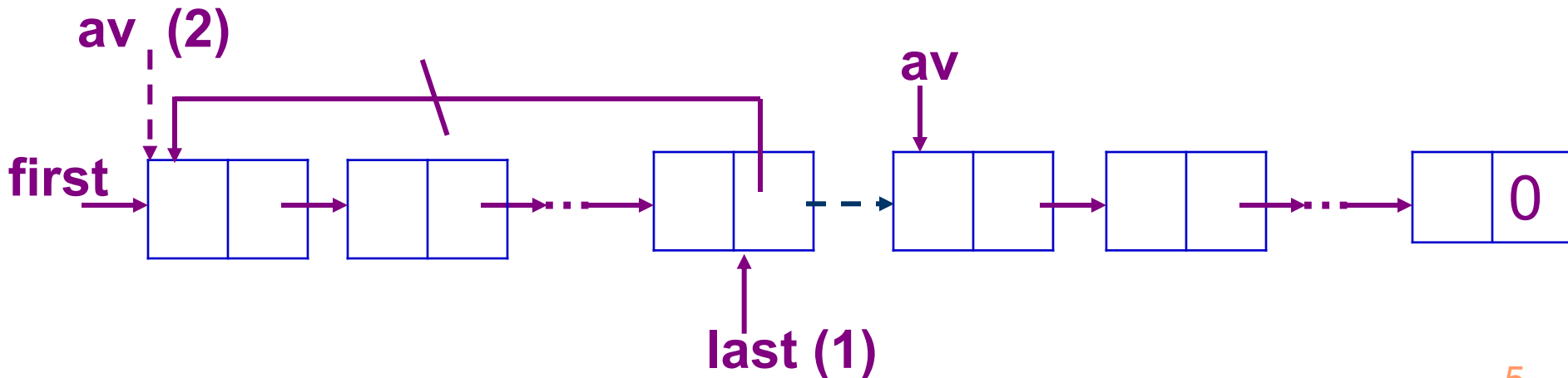
A circular list may be deleted in $O(1)$:

```
template <class T>
void CircularList<T>::~~CircularList()
{ // delete the circular list.
    if (last) { ChainNode <T> * first = last→link;
        last→link = av; // (1)
        av = first; // (2)
        last = 0;
    }
}
```



A circular list may be deleted in $O(1)$:

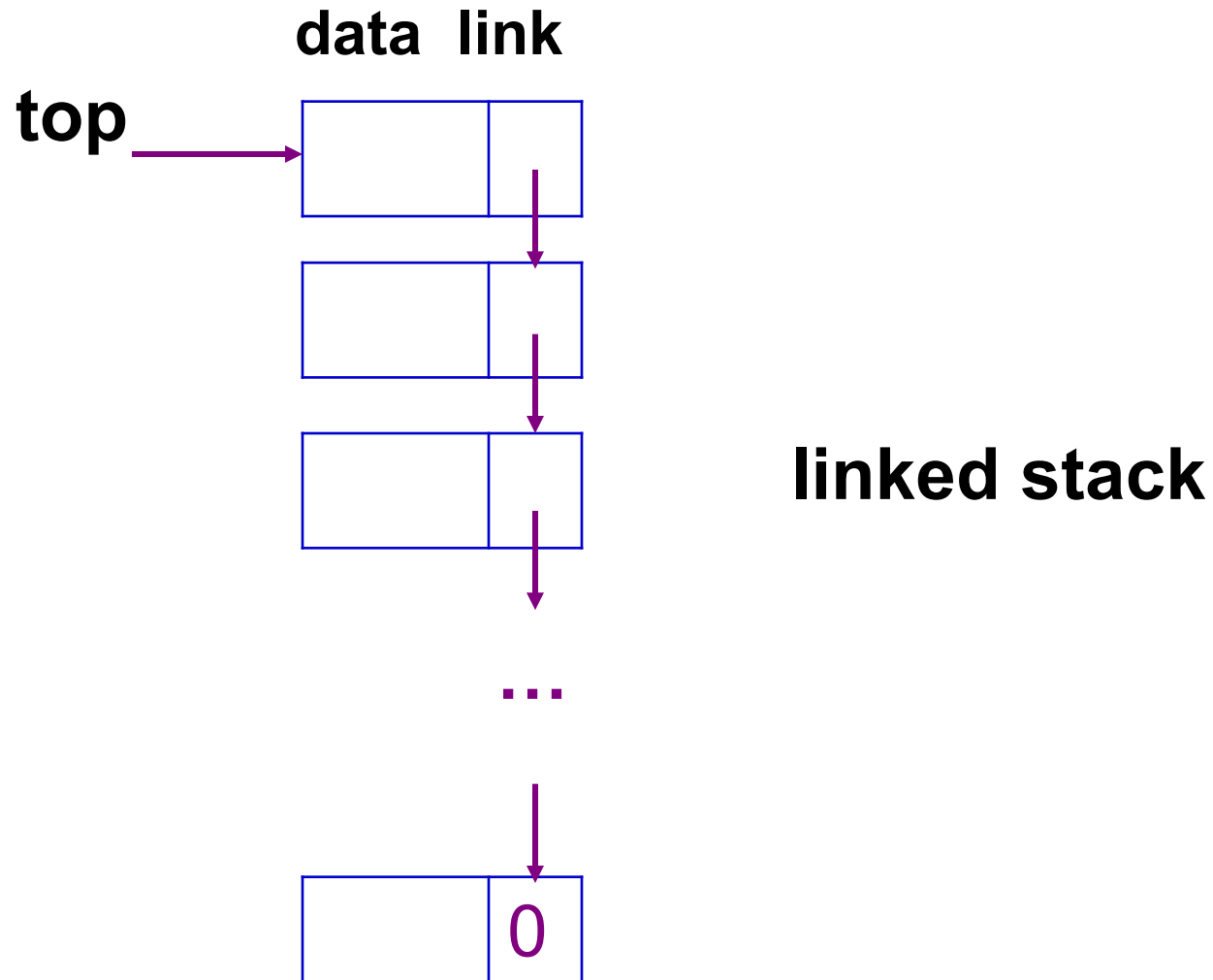
```
template <class T>
void CircularList<T>::~~CircularList()
{ // delete the circular list.
    if (last) { ChainNode <T> * first = last→link;
        last→link = av; // (1)
        av = first; // (2)
        last = 0;
    }
}
```



A chain may be deleted in $O(1)$ if we know its **first** and **last** nodes:

```
template <class T>
Chain<T>::~~Chain()
{ // delete the chain
    if (first) {
        last→link = av;
        av = first;
        first = 0;
    }
}
```

4.6 Linked Stacks and Queues



Assume the **LinkedStack** class has been declared as **friend** of **ChainNode<T>**.

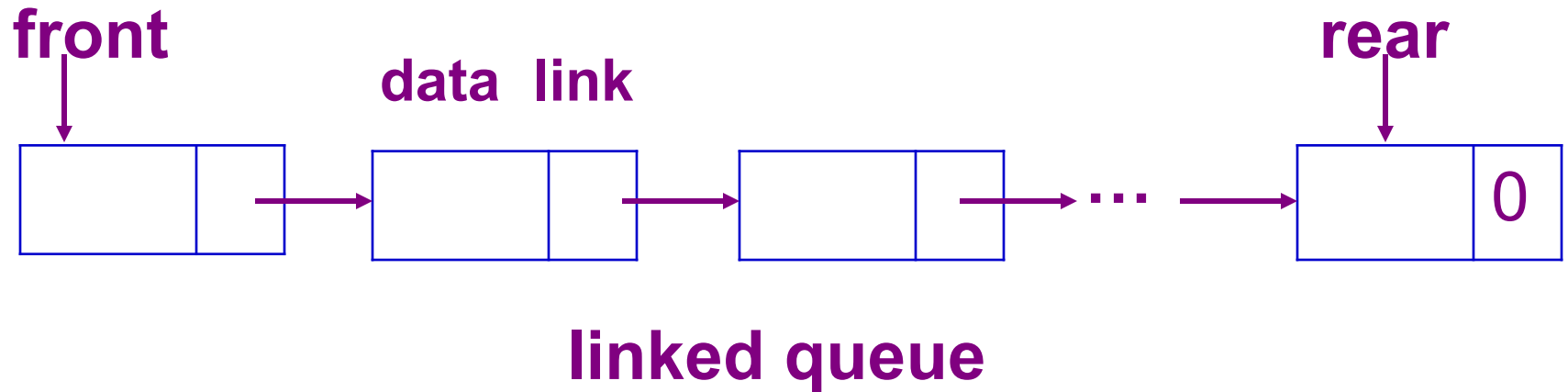
```
template <class T>
class LinkedStack {
public:
    LinkedStack() { top=0;}; // constructor initializing top to 0
    // LinkedStack manipulation operations
    ...
private:
    ChainNode<T> *top;
};
```



```
template <class T>
void LinkedStack<T>::Push(const T& e) {
    top = new ChainNode(e, top);
}
```

```
template <class T>
void LinkedStack<T>::Pop()
{ // delete top node from the stack.
    if (IsEmpty()) throw “Stack is empty. Cannot delete.”;
    ChainNode<T> * delNode = top;
    top = top→link;
    delete delNode;
}
```

The functions *IsEmpty* and *Top* are easy to implement, and are omitted.



The functions of **LinkedQueue** are similar to those of **LinkedStack**, and are left as exercises.

4.7 Polynomials

4.7.1 Polynomial Representation

Since a polynomial is to be represented by a list, we say Polynomial is **IS-IMPLEMENTED-IN-TERMS-OF** List.

Definition: a data object of Type A **IS-IMPLEMENTED-IN-TERMS-OF** a data object of Type B if the Type B object is central to the implementation of Type A object. ---Usually by declaring the Type B object as a data member of the Type A object.

$$A(x) = a_m x^{e_m} + a_{m-1} x^{e_{m-1}} + \dots + a_1 x^{e_1}$$

Where $a_i \neq 0$, $e_m > e_{m-1} > \dots, e_1 \geq 0$

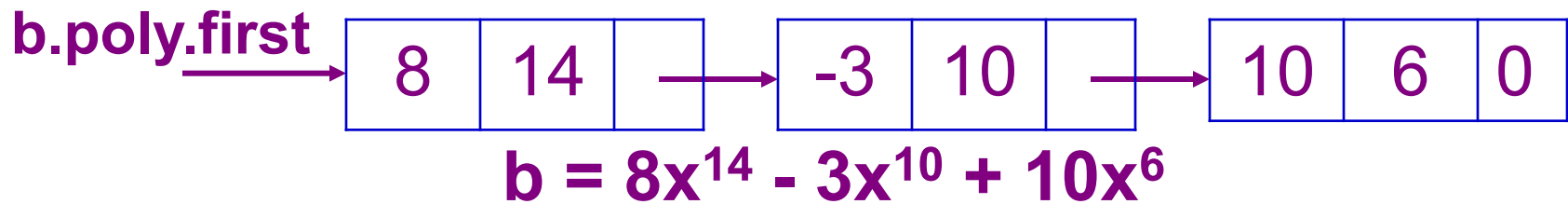
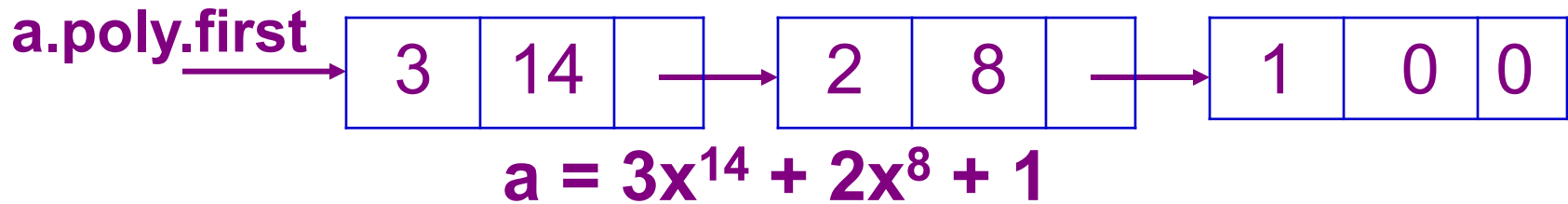
- Make the chain **poly** a data member of **Polynomial**.
- Each **ChainNode** will represent a term. The template **T** is instantiated to struct **Term**:

```
struct Term
{ // all members of Term are public by default
    int coef;
    int exp;
    Term Set(int c, int e) { coef=c; exp=e; return *this;};
};
```

```

class Polynomial {
public:
    // public functions defined here
private:
    Chain<Term> poly;
};

```



4.7.2 Adding Polynomials

To add two polynomials **a** and **b**, use the chain iterators **ai** and **bi** to move along the terms of a and b.

```
1 Polynomial Polynomial::operator+ (const Polynomial& b) const
2 { // *this (a) and b are added and the sum returned
3   Term temp;
4   Chain<Term>::ChainIterator ai = poly.begin(),
5                               bi = b.poly.begin();
6   Polynomial c;
```

```
7  while (ai != poly.end() && bi != b.poly.end()) { //not null
8      if (ai→exp == bi→exp) {
9          int sum = ai→coef + bi→coef;
10         if (sum) c.poly.InsertBack(temp.Set(sum, bi→exp);
11         ai++; bi++; // to next term
12     }
13     else if (ai→exp < bi→exp) {
14         c.poly.InsertBack(temp.Set(bi→coef, bi→exp));
15         bi++; // next term of b
16     }
17     else {
18         c.poly.InsertBack(temp.Set(ai→coef, ai→exp));
19         ai++; // next term of a
20     }
21 }
```

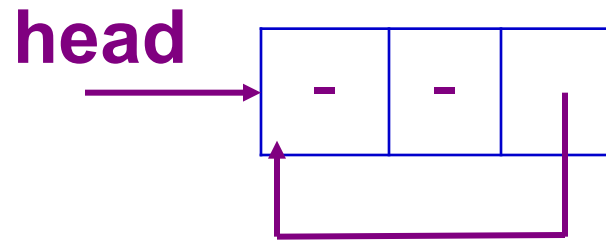
```
22  while (ai != poly.end()) { // copy rest of a
23      c.poly.InsertBack(temp.Set(ai→coef, ai→exp));
24      ai++;
25  }
26  while (bi != b.poly.end()) { // copy rest of b
27      c.poly.InsertBack(temp.Set(bi→coef, bi→exp));
28      bi++;
29  }
30  return c;
31 }
```

Analysis:

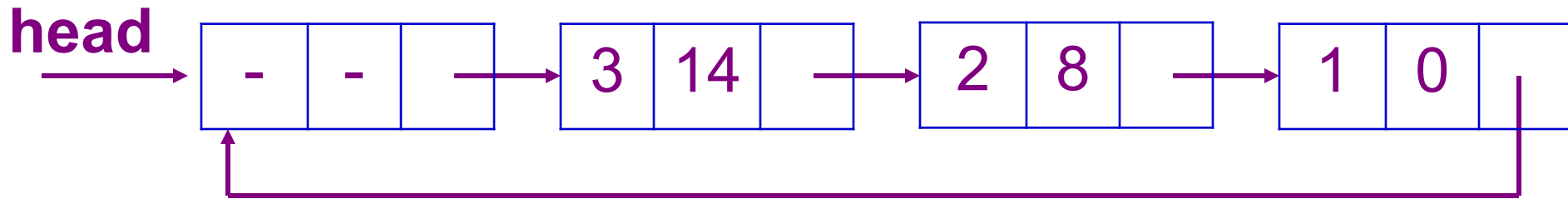
Assume a has **m** terms, b has **n** terms. The computing time is $O(m+n)$.

4.7.3 Circular List Representation of Polynomials

Polynomials represented by circular lists with head node are as in the next slide:



(a) Zero polynomial



(b) $3x^{14} + 2x^8 + 1$

Adding circularly represented polynomials

- The **exp** of the head node is set to -1 to push the rest of **a** or **b** to the result.
- Assume the **begin()** function for class **CircularListWithHead** return an iterator with its current points to the node **head→link**.

```

1 Polynomial Polynomial::operator+(const Polynomial& b) const
2 { // *this (a) and b are added and the sum returned
3   Term temp;
4   CircularListWithHead<Term>::Iterator ai = poly.begin(),
5                                     bi = b.poly.begin();
6   Polynomial c; //assume constructor sets head→exp = -1
7   while (1) {
8     if (ai→exp == bi→exp) {
9       if (ai→exp == -1) return c;
10      int sum = ai→coef + bi→coef;
11      if (sum) c.poly.InsertBack(temp.Set(sum, ai→exp);
12      ai++; bi++; // to next term
13    }

```

```
14  else if (ai→exp < bi→exp) {
15      c.poly.InsertBack(temp.Set(bi→coef, bi→exp));
16      bi++; // next term of b
17  }
18  else {
19      c.poly.InsertBack(temp.Set(ai→coef, ai→exp));
20      ai++; // next term of a
21  }
22 }
23 }
```

4.10 Doubly Linked Lists

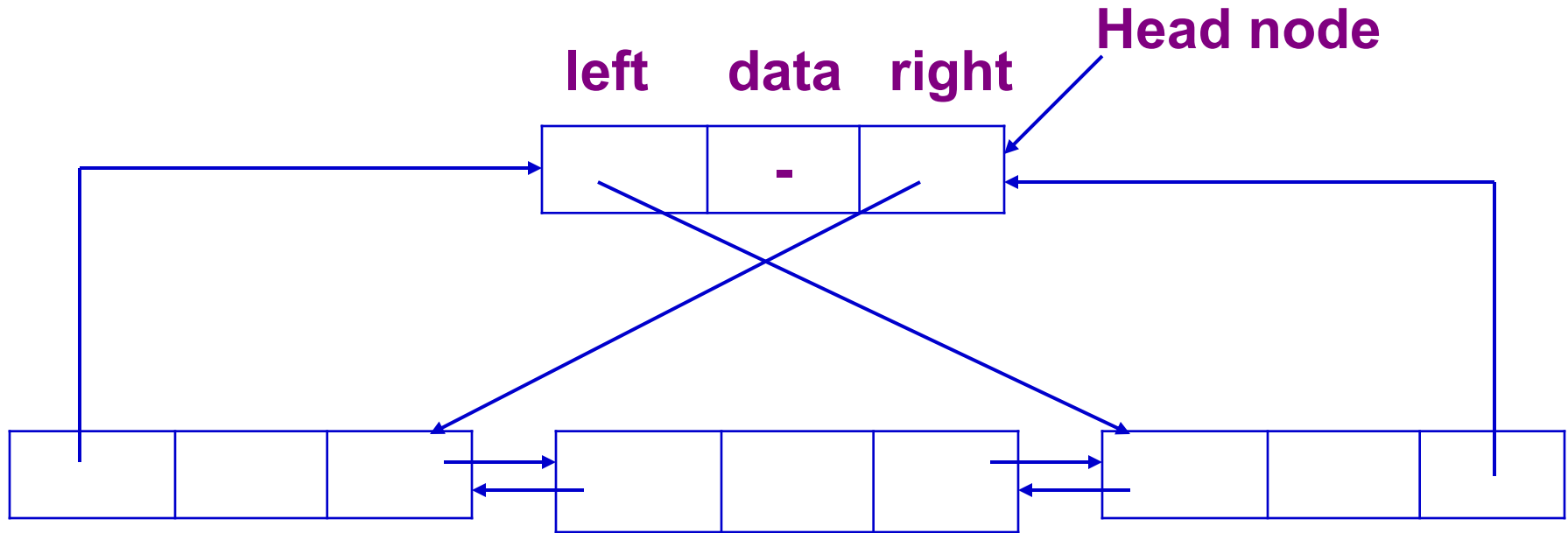
Difficulties with singly linked list:

- can easily move only in one direction
- not easy to delete an arbitrary node
 - requires knowing the preceding node

A node in doubly linked list has at least 3 field: data, left and right, this makes moving in both directions easy.



A doubly linked list may be circular. The following is a doubly linked circular list with head node:



Suppose p points to any node, then
 $p == p \rightarrow \text{left} \rightarrow \text{right} == p \rightarrow \text{right} \rightarrow \text{left}$

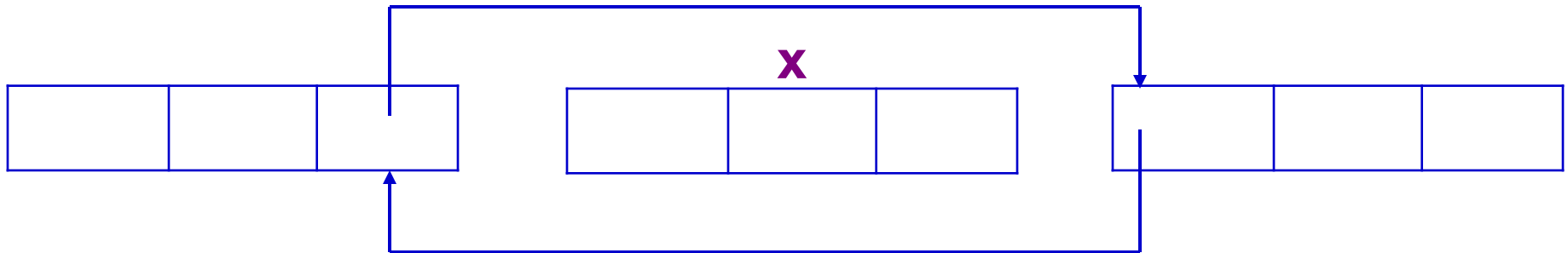
```
class DbList;

class DbListNode {
friend class DbList;
private:
    int data;
    DbListNode *left, *right;
};

class DbList {
public:
    // List manipulation operations
    ...
private:
    DbListNode *first; // points to head node
};
```


Delete

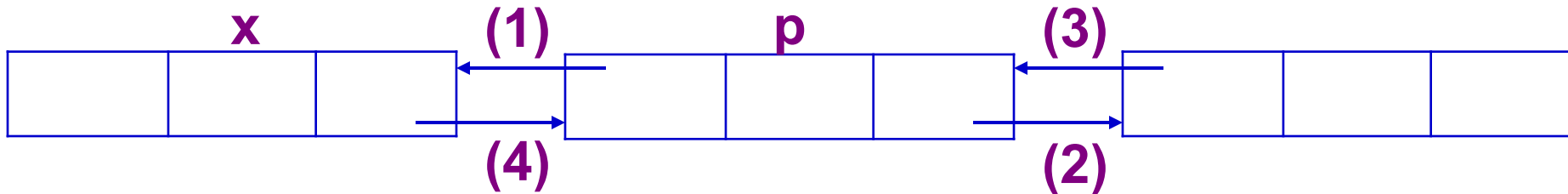
```
void DbList::Delete(DblListNode *x )
{
    if(x == first) throw "Deletion of head node not permitted";
    else {
        x→left→right = x→right;
        x→right→left = x→left;
        delete x;
    }
}
```



Insert

```
void DbList::Insert(DbListNode *p, DbListNode *x )  
{ // insert node p to the right of node x
```

```
    p→left = x;           // (1)  
    p→right = x→right;    // (2)  
    x→right→left = p;     // (3)  
    x→right = p;          // (4)  
}
```



Others

- General List
 - ◆ $A(B(), 25, C(D(1, 30), 34))$
 - ◆ Linked Representation
 - ◆ Depth
- Linked Representation for Sparse Matrix

Exercises: P223-6(f,h)

1. Given a sorted single linked list $L = \langle a_1, \dots, a_n \rangle$, where $a_i.data \leq a_j.data$ ($i < j$).

Try to write an algorithm of inserting a new data element X to L , and analyze its complexities.

2. Given a linear list $L = \langle a_1, \dots, a_n \rangle$, implemented by a single linked list.

Delete data a_i with Time Complexity $O(1)$. We have a pointer to $node(a_i)$.